

Revista PROGRAMAR

ARTIGOS > TEMA DE CAPA

TRAVESSIA DE UMA ÁRVORE DE DIRETÓRIOS USANDO RECURSIVIDADE

23 DE DEZEMBRO DE 2015 | PATRICIO DOMINGUES CARLOS GRILO VÍTOR CARREIRA

Introdução

O diretório é um elemento familiar nos sistemas de ficheiros, sendo empregue para organizar o armazenamento de dados em suporte persistente como, por exemplo, discos rígidos. Uma operação relativamente comum num sistema de ficheiros é a travessia da árvore de diretórios, através da qual são visitados todos os subdiretórios e ficheiros. Este artigo foca o uso de recursividade e das funções `stat` e `readdir` para a travessia de uma árvore de diretório recorrendo à metodologia denominada de *busca em profundidade* (*depth-first search* na designação anglo-saxónica) (Knuth, 1968) (Wikipedia, 2015). Embora o artigo assente no ambiente Linux e na linguagem C, a metodologia utilizada pode ser empregue, com algumas adaptações, noutras plataformas e com outras linguagens de programação.

Diretórios, caminhos absolutos e caminhos relativos

Um diretório pode conter ficheiros, outros diretórios e assim sucessivamente, resultando numa estrutura em árvore. O ponto de entrada da árvore é designado por diretório **raiz**, sendo representado pela barra `/` (*slash bar* na designação anglo-saxónica) em sistemas operativos UNIX e derivados. A barra `/` é ainda empregue como separador na representação de caminhos dos sistemas de ficheiros que identifiquem um determinado recurso, seja um diretório ou um ficheiro. No que respeita a caminhos, esses podem ser representados de forma absoluta ou relativa. A representação absoluta requer que o caminho do recurso que se pretende identificar seja especificado desde a raiz, iniciando-se pela barra `/`. Por exemplo, o caminho absoluto `/bin/ls` identifica a localização tradicional do comando `ls` num sistema UNIX. Por sua vez, e como o nome sugere, um caminho relativo é especificado em relação ao diretório corrente,

não se iniciando, portanto, com a barra `/`. Por exemplo, o caminho `programar/artigo.doc` refere-se ao ficheiro `artigo.doc` que se encontra no diretório `programar` que existe no diretório corrente. A Listagem 1 mostra o primeiro nível da árvore de diretórios definida pelo sistema de ficheiros de um sistema Linux.

```
|— bin
|— boot
|— dev
|— etc
|— home
|— lib
|— lost+found
|— media
|— mnt
|— opt
|— proc
|— root
|— run
|— sbin
|— srv
|— sys
|— tmp
|— usr
|— var
```

Listagem 1: árvore de diretórios de um sistema Linux a partir do diretório `/` (saída do `tree: tree -L 1 -d /`)

As funções da família *stat*

As funções da família `stat` – `stat`, `fstat` e `lstat` – permitem o acesso aos metadados de uma entrada do sistema de ficheiros, seja um diretório, um ficheiro, uma ligação simbólica ou um outro recurso. A designação metadados abrange os elementos caracterizadores da entrada, tal como a data de criação, as permissões, a identificação do dono e grupo do recurso, entre outros elementos. Para o efeito, as funções da família `stat` recorrem à estrutura `struct stat` para representação dos metadados. A `struct stat` tem 13 campos, apresentados na Listagem 2 que foi diretamente extraída da saída do comando `man 2 stat`. Note-se que algumas versões de UNIX podem ter campos adicionais na `struct stat`, sendo que o uso desses campos extra num programa limita necessariamente a portabilidade do código.

```

struct stat {
    dev_t      st_dev;      /* ID of device containing file
*/
    ino_t      st_ino;      /* inode number */
    mode_t     st_mode;     /* protection */
    nlink_t    st_nlink;    /* number of hard links */
    uid_t      st_uid;      /* user ID of owner */
    gid_t      st_gid;      /* group ID of owner */
    dev_t      st_rdev;     /* device ID (if special file)
*/
    off_t      st_size;     /* total size, in bytes */
    blksize_t  st_blksize;  /* blocksize for filesystem I/O

```

Listagem 2: estrutura `struct stat` (fonte: `man 2 stat`)

A Listagem 3 apresenta o código da aplicação `metadados_dir_corrente` que exemplifica o uso da função `stat`, mostrando na saída padrão o valor dos 13 campos de metadados da `struct stat` para a entrada `.` (ponto). A entrada `.` é uma entrada especial que representa o diretório corrente. Outra entrada especial é o `..` (ponto ponto) que representa o diretório pai ou diretório ascendente. Tanto o diretório `.` e o diretório *pai* `..` existem em todos os diretórios, pelo que as entradas `.` e `..` são sempre válidas (no caso do diretório raiz `/`, a entrada `..` aponta para o próprio diretório raiz). A Listagem 4 mostra a saída resultante da execução da aplicação `metadados_dir_corrente`. Obviamente, a saída varia consoante o diretório corrente onde é executada a aplicação.

Em termos de código, para além da chamada à função `stat`, a aplicação `metadados_dir_corrente` implementa a função `tipo_file_str` que recebendo o campo `st_mode` da `struct stat` devolve uma cadeia de caracteres indicando se a entrada corresponde a um ficheiro regular, diretório, atalho (*symbolic link*) ou “outro”. Para determinar o tipo de entrada, a função `tipo_file_str()` faz uso das *macros* do sistema `S_IS_REG()`, `S_IS_DIR()` e `S_IS_LNK()` indicando como parâmetro o campo `st_mode` da entrada em análise. Importa notar que para além do tipo de entrada, o campo `st_mode` contém, a informação referente às permissões de acesso do recurso – leitura, escrita, execução – para o dono, o grupo e o *outros*. Finalmente, os campos do tipo `time_t`, isto é, `st_atime`, `st_ctime` e `st_mtime` são convertidos para uma representação dia/hora através da função `ctime_r` que corresponde à versão *reentrante* da função `ctime`. Mais detalhes podem ser encontrados na respetiva página do manual (`man ctime`).

Conforme referido anteriormente, a família de funções `stat` tem, para além da função `stat` propriamente dita, as funções `fstat` e `lstat`. Na função `fstat`, o ficheiro do qual se pretende os respetivos metadados é indicado por um descritor de ficheiro, descritor esse previamente obtido através da função

`open`. Por sua vez, a função `lstat` comporta-se de forma similar à função `stat`, com uma exceção: quando é indicada uma ligação simbólica, o `lstat` devolve os metadados da ligação simbólica e não os metadados para a qual aponta a ligação simbólica. Mais adiante neste artigo, far-se-á uso da função `lstat` na implementação recursiva da travessia de uma árvore de diretórios.

```
#include <...>

char *tipo_file_str(mode_t st_mode){
    if( S_ISREG(st_mode) ){
        return "ficheiro normal";
    }else if( S_ISDIR(st_mode) ){
        return "Diretório";
    }else if( S_ISLNK(st_mode) ){
        return "Atalho";
    }
    /* ainda aqui? */
    return "outro";
}
```

Listagem 3: aplicação `mostra_metadados_dir_corrente`

```
st_dev = 8.17
st_ino = 3662149
st_mode = 41fd (Diretório)
st_nlink = 2
st_uid = 1000
st_gid = 1000
st_rdev= 0
st_size= 4096
st_blksize= 4096
st_blocks=8 (blocos de 512B)
st_atime (ultimo acesso)=Sat Oct 31 01:04:05 2015
st_mtime (ultima modificação)=Sat Oct 31 01:10:16 2015
st_ctime (ultima alter. estado)=Sat Oct 31 01:10:16 2015
```

Listagem 4: saída produzida pela execução de `metadados_dir_corrente`

Acesso ao conteúdo de um diretório

Em termos de programação, o acesso ao conteúdo de um diretório – ficheiros, diretórios, atalhos, etc. – apresenta algumas semelhanças com a leitura de um ficheiro, sendo necessário efetuar a abertura do diretório antes que se possa aceder ao conteúdo e fechar o diretório quando se terminou o respetivo uso. Para o efeito existem as funções `opendir` (abertura) e `closedir` (fecho). O acesso ao conteúdo é feito através de chamadas sucessivas à função `readdir`

que devolve uma entrada por chamada. Os protótipos das funções necessárias para iterar o conteúdo de um diretório estão indicados na Listagem 5.

No acesso ao conteúdo de um dado diretório, a primeira operação consiste na abertura do diretório através da função `opendir` que, caso seja bem sucedida, devolve o endereço de memória de um descritor de diretório do tipo de dados `DIR`.

```
#include <sys/types.h>
#include <dirent.h>

DIR *opendir(const char *name);

struct dirent *readdir(DIR *dirp);
int readdir_r(DIR *dirp, struct dirent *entry,
              struct dirent **result);

int closedir(DIR *dirp);
```

Listagem 5: protótipos das funções `opendir`, `readdir` e `closedir` (fonte: `man opendir` e `man readdir`)

O descritor devolvido pela função `opendir` é empregue para iterar o conteúdo do diretório através da função `readdir`. Esta função deve ser chamada em ciclo, devolvendo para cada chamada uma entrada do diretório. A função `readdir` assinala o término da iteração através da devolução do valor `NULL`, indicando que já foram iteradas todas as entradas do diretório. O passo final corresponde ao encerrar do diretório, efetuando-se através da chamada à função `closedir`. De seguida, analisam-se separadamente cada uma das etapas para acesso aos conteúdos de um diretório: abertura, leitura e encerramento do diretório.

Obtenção do descritor – `opendir`

A obtenção de um descritor do tipo `DIR` para um determinado diretório efetua-se através da função `opendir`. A função recebe como único parâmetro o caminho (absoluto ou relativo) do diretório para o qual se pretende receber um descritor. Caso a operação seja bem sucedida, a função devolve o endereço de uma estrutura do tipo `DIR`. Essa estrutura contém, entre outros elementos, um descritor para o diretório. Por questões de simplicidade, este artigo emprega a designação *descritor de diretório* para designar a estrutura do tipo `DIR`. Caso ocorra um erro na tentativa de abertura do diretório (por exemplo, o diretório especificado não existe), a função `opendir` devolve `NULL`, sendo atribuído à variável de erro `errno` o respetivo código numérico do erro. Dado que os códigos

gos numéricos são pouco descritivos para os operadores humanos, é possível obter uma frase descritiva do código de erro indicado pelo valor da variável `errno` através da função `strerror`. A Listagem 6 apresenta o código do programa `mostra_dir_corrente` que faz uso da função `strerror` para tornar as mensagens de erro mais compreensíveis. Mais informações sobre a variável `errno` e a função `strerror` estão disponíveis nas respetivas páginas de manual eletrónico (`man errno` e `man strerror`).

Iteração do diretório – `readdir`

Obtido um descritor para o diretório, passa-se à fase de leitura das entradas existentes no diretório. Para o efeito faz-se uso da função `readdir` que atua de forma similar a um operador de iteração: sempre que é chamada, devolve uma entrada diferente do diretório até que estejam esgotadas todas as entradas. Deste modo, e caso se pretenda iterar todo o conteúdo do diretório, a função `readdir` deve ser chamada em ciclo até que seja alcançada a última entrada do diretório.

Como parâmetro, a função `readdir` recebe o descritor de diretório anteriormente obtido através da função `opendir`. Caso a chamada seja bem sucedida, a função `readdir` devolve o endereço de uma estrutura do tipo `struct dirent` associada à entrada corrente. Ao invés, se já tiverem sido percorridas todas as entradas do diretório, a função `readdir` devolve `NULL`, devolvendo ainda `NULL` na ocorrência de um erro, por exemplo, caso o descritor de diretório indicado seja inválido. Deste modo, torna-se necessário distinguir entre o término do diretório (situação normal) e a ocorrência de um erro (situação anormal). A distinção é feita através do valor da variável de erro `errno` que fica com um valor não nulo caso tenha ocorrido um erro na execução da função `readdir`. O código da função `mostra_dir` (Listagem 6) exemplifica o uso da variável `errno` para distinguir entre a normalidade do término do diretório e a anormalidade que é a ocorrência de erro na execução da função `readdir`.

De acordo com a norma POSIX é sempre garantido que a `struct dirent` tenha o campo `d_name`, que como o nome sugere, contém o nome da entrada. No Linux, a `struct dirent` contém mais alguns campos, nomeadamente o campo `d_type` que permite determinar o tipo da entrada (diretório, ficheiro regular, ligação simbólica, etc.). Contudo, caso se pretenda manter a portabilidade do código para ambientes POSIX, apenas deve ser empregue o campo `d_name`. Este artigo segue esta abordagem, fazendo uso da função `stat` para aceder aos metadados de uma entrada devolvido via `struct dirent` pela função `readdir`. A `struct dirent` disponibilizada em sistemas Linux é apresentada na Listagem 7.

```
#include <stdio.h>
#include <sys/types.h>
#include <dirent.h>
#include <errno.h>
#include <string.h>
#include <stdlib.h>

/*
 * Mostra as entradas do diretório 'nome_dir'.
 * Devolve 0 se tudo ok, -1 em caso de erro.
 */
int mostra_dir(const char *nome_dir){
```

Listagem 6: programa `mostra_dir_corrente`

```
struct dirent {
    ino_t    d_ino;        /* inode number */
    off_t    d_off;        /* not an offset */
    unsigned short d_reclen; /* length of record */
    unsigned char d_type; /* type of file;
                           not supported by all filesystem types */
    char      d_name[256]; /* filename */
};
```

Listagem 7: `struct dirent` em sistemas Linux (fonte: `man readdir`)

Encerramento

O encerramento de um descritor `DIR` é realizado através da chamada à função `closedir`. Essa operação é necessária para garantir que os recursos associados ao descritor são convenientemente libertados. A função `closedir` recebe o descritor do tipo `DIR` que se pretende encerrar. Caso o encerramento seja bem sucedido, a função devolve 0; caso contrário, devolve -1, atribuindo à variável `errno` um código numérico descritivo do erro. Na função `mostra_dir` (Listagem 7) exemplifica-se a verificação da eventual ocorrência de erros na sequência do fecho do diretório.

Uso combinado das funções `readdir` e `stat`

A travessia de uma árvore de diretórios requer a capacidade de identificar os tipos de cada entrada existente num diretório. De facto, torna-se necessário identificar as entradas do tipo diretório por forma a que esses subdiretórios possam eles também ser visitados. Para o efeito, torna-se necessário o uso

combinado das funções `readdir` (convenientemente enquadrada com `opendir` e `closedir`) e `stat`. A Listagem 8 apresenta o código da aplicação `mostra_dir_stat` que faz uso das funções `readdir` e `stat` para listar o conteúdo do diretório cujo nome (absoluto ou relativo) é indicado à aplicação através do único argumento a linha de comando. A aplicação lista o conteúdo do diretório indicado, mostrando o tipo – diretório, ficheiro regular, ligação simbólica – de cada entrada. Assim, para cada entrada do diretório devolvida pela função `readdir` é chamada a função `stat` usando como parâmetro o campo `d_name` da `struct dirent` devolvido pela função `readdir`. Seguidamente, o campo `st_mode` da `struct stat` preenchido pela função `stat` é passado à função `devolve_tipo_entrada` que devolve uma cadeia de caracteres descritiva do tipo de entrada, tais como `DIR` para diretório, `ligação simbólica` para uma ligação simbólica, `ficheiro` para um ficheiro regular, etc. Importa notar que a determinação do tipo de entrada é feita aplicando-se uma operação de *AND* binário entre o valor `st_mode` e a máscara `S_IFMT`, comparando-se o resultado dessa operação com as constantes `S_IFDIR` (diretório), `S_IFIFO` (*pipe FIFO*), `S_IFLNK` (ligação simbólica), `S_IFREG` (ficheiro regular) e `S_IFSOCK` (*socket*). Tanto a máscara `S_IFMT`, bem como as constantes `S_IFIFO`, `S_IFLNK`, `S_IFREG` e `S_IFSOCK` são disponibilizadas quando se faz uso dos ficheiros de cabeçalho (`.h`) associados à função `stat`. Esta informação e mais alguns detalhes constam da página do manual eletrónico da função `stat`, acessível através de `man stat`.

```
#include <...>
/* Devolve string descritiva da entrada st_mode. */
char *devolve_tipo_entrada(mode_t st_mode){
    switch( st_mode & S_IFMT ){
        case S_IFDIR:
            return "DIR";
        case S_IFIFO:
            return "FIFO/pipe";
        case S_IFLNK:
            return "ligação simbólica";
        case S_IFREG:
            return "ficheiro";
    }
```

Listagem 8: programa `mostra_dir_stat`

Travessia de uma árvore de diretórios

Analisando-se uma árvore de diretório, facilmente se identifica um padrão repetitivo: uma árvore é composta por subárvores. De facto, um subdiretório pode ser considerado a raiz de uma subárvore do diretório que contém o subdiretório e assim sucessivamente. Deste modo, a travessia de um diretório e de todos os

seus subdiretórios pode ser decomposta noutras operações de travessia, uma travessia por cada subdiretório. Na informática, este tipo de operação que se subdivide em operações do mesmo tipo em sub-conjuntos do conjunto original de dados presta-se ao uso de recursividade. De uma forma aproximada, pode dizer-se que em termos de programação programáticos, a recursividade consiste no chamar de uma função por ela própria por forma a tratar um subconjunto dos dados originais. O chamar da função por ela própria origina uma nova instância da função. No caso da travessia de uma árvore de diretórios, cada nova instância da função de travessia recebe um subdiretório do diretório que está a ser processado pela função chamante, invocando ela própria uma nova instância para cada subdiretório que tenha e assim sucessivamente.

Embora a descrição seja relativamente complexa, em termos de código, a operacionalização é simples: sempre que se deteta uma entrada do tipo *diretório* no diretório corrente, cria-se uma nova instância da função. Essa nova instância (i.e., chamada recursiva) recebe como parâmetro na chamada à função o caminho correspondente ao diretório corrente acrescido do nome do diretório a ser processado pela nova instância. Por exemplo, caso a função se encontre a processar o diretório `dir_A/dir_B/dir_C` e se torne necessário processar o subdiretório `dir_D`, a função é chamada com o caminho `dir_A/dir_B/dir_C/dir_D`. Quando termina a travessia desse subdiretório (e dos eventuais subsubdiretórios, etc.), a nova instância termina, regressando a execução ao diretório corrente para processamento da entrada seguinte e assim sucessivamente.

A Listagem 9 apresenta o código da função `dir_recursivo` que efetua a travessia uma árvore de diretórios e subdiretórios. O primeiro parâmetro – `nome_dir` – corresponde ao caminho do diretório a percorrer. Por sua vez, o segundo parâmetro – `profundidade` – indica a profundidade do diretório em análise. A profundidade mede o número de subdiretórios a que se encontra o diretório corrente em relação ao diretório original. Por exemplo, se a função for chamada com o diretório original `/` (raiz) num sistema Linux, o subdiretório `/usr` terá profundidade 1 e o subsubdiretório `/usr/bin` terá profundidade 2 e assim sucessivamente. Para o efeito, considera-se que o diretório original tem profundidade 0 (zero), sendo esse o valor que deve ser passado na primeira chamada à função `dir_recursivo`. Depois, para cada chamada recursiva, é passado o valor de profundidade + 1, refletindo o facto de que a nova instância da função processa um novo nível de subdiretório.

Na travessia recursiva de uma árvore de diretórios é necessário ainda ter em conta duas situações particulares. A primeira está relacionada com os diretórios `.` (ponto) e `..` (ponto ponto), que conforme anteriormente foi visto, correspondem respetivamente ao diretório corrente e ao diretório *pai*. Dadas as especificidades desses diretórios, torna-se necessário ignorá-los em termos de tra-

vessia recursiva. De facto, caso se considerasse o diretório `.` para efeitos de travessia, chamando recursivamente a função `dir_recursivo` com `.`, cair-se-ia num ciclo sem fim, preso no diretório corrente. Na prática, a aplicação terminaria devido a um excessivo nível de recursividade materializado com um transbordo de pilha ou *stack overflow* na designação anglo-saxónica (Chris Anley, 2007). Similarmente, o diretório `..` deve ser ignorado, pois levaria o controlo de execução para o diretório ascendente, impedindo o normal processamento do diretório corrente e respetivos subdiretórios. O ignorar dos diretórios `.` e `..` para efeitos de travessia consegue-se comparando o nome do novo diretório a visitar com `.` e `..`, comparação efetuada através da função `strcmp`.

A segunda situação a acautelar está relacionada com ligações simbólicas. Assim, caso seja detetada uma ligação simbólica para um diretório, essa mesma deve ser ignorada, caso contrário corre-se o risco de se saltar para outra árvore de diretório, interrompendo a travessia pretendida. A necessidade de deteção de uma ligação simbólica requer o uso da função `lstat` que, conforme anteriormente referido, no caso de uma ligação simbólica devolve os metadados referentes à ligação simbólica e não ao recurso (diretório/ficheiro) apontado pela ligação simbólica. Desta forma, torna-se possível detetar uma ligação simbólica.

O programa `dir_recursivo` mostrado na Listagem 9 efetua a travessia de todos os diretórios, ficheiros e subdiretórios de uma árvore de diretório. A raiz da árvore de diretórios deve ser indicada como o único parâmetro da linha de comandos do programa. Como é óbvio, apenas são mostrados os conteúdos para os quais o utilizador que executa a aplicação tem permissões de acesso.

```
#include <...>
/* Devolve string descritiva da entrada
 * indicada por st_mode. Caso o tipo de entrada não
 * seja reconhecido, a função devolve "outro". */
char *devolve_tipo_entrada(mode_t st_mode){
    switch( st_mode & S_IFMT ){
        case S_IFDIR:
            return ("DIR");
        case S_IFIFO:
            return ("FIFO/pipe");
        case S_IFLNK:
            return ("ligação simbólica");
```

Listagem 9: programa `dir_recursivo`

Reentrância

O facto do código apresentado neste artigo usar a função não reentrante `readdir`, leva a que não seja seguro usar o código em ambiente multithreaded. Contudo, no caso da função `readdir` é relativamente fácil substituir o seu uso pela versão reentrante `readdir_r`. Esta abordagem é deixada como exercício para os leitores, sugerindo-se a consulta do manual eletrónico (`man readdir_r`).

Nota final

Este artigo abordou a travessia com busca em profundidade de uma árvore de diretórios, com o intuito de visitar todos os recursos (ficheiros, subdiretórios) dessa mesma árvore. Embora o enfoque tenha sido em diretórios, uma metodologia similar pode ser empregue, com as devidas adaptações, com outros tipos de árvores, por exemplo, árvores binárias.

Bibliografia

1. Chris Anley, J. H. (2007). *The Shellcoder's Handbook: Discovering and Exploiting Security Holes (2nd ed.)*. Wiley Publishing, Inc.
2. Knuth, D. (1968). *The Art of Computer Programming, volume 1: Fundamental Algorithms*. Addison-Wesley.
3. Wikipedia. (2015). *Busca em Profundidade*. Obtido em 06 de 11 de 2015, de Wikipedia: https://pt.wikipedia.org/wiki/Busca_em_profundidade



Publicado na edição 51 (PDF) da Revista PROGRAMAR.



Patrício Domingues

Patrício Domingues é doutorado em Engenharia Informática e professor do Departamento de Eng^a Informática na Escola Superior de Tecnologia e Gestão (ESTG) do Instituto Politécnico de Leiria (IPLeiria). Tem lecionado, entre outras, a disciplina de Programação Avançada da Licenciatura em Engenharia Informática. É ainda responsável pelo GPU Education Center (antigo NVIDIA CUDA Teaching) da ESTG/IPLeiria.



Carlos Grilo

Carlos Grilo é coordenador do Mestrado em Engenharia Informática – Computação Móvel da Escola Superior de Tecnologia e Gestão do Politécnico de Leiria. Leciona as disciplinas de Programação Avançada e Desenvolvimento de Aplicações Empresariais ao Curso de Licenciatura em Engenharia Informática.



Vítor Carreira

Vítor Carreira leciona as disciplinas de Programação Avançada, Sistemas Operativos e Desenvolvimento de Aplicações Distribuídas ao curso de Licenciatura em Engenharia Informática da Escola Superior de Tecnologia e Gestão do Politécnico de Leiria.

